

A LECTURE ON

LLM Observability.

Seeing inside a running system.

Where we are.

- You can build the system.
-
- You can score the output (evals).
-
- You cannot see what happened inside a run.

Three questions.

- Why did this take 9 seconds?
 - Why did it cost 11k tokens?
 - Why did step 4 return garbage?
-

Part 1 — The Problem

The problem.

Flat logs lose the tree.

print() breaks down.

- Nested calls.
- Async / concurrent.
- Multi-step agent loops.
- Flat logs lose the tree.

What we actually need.

— Structured.

— Causal — parent $\rightarrow$ child.

— Queryable.

Part 2 — OpenTelemetry

OpenTelemetry.

The vocabulary that outlives the vendor.

Trace.

One request's full lifecycle.

Span.

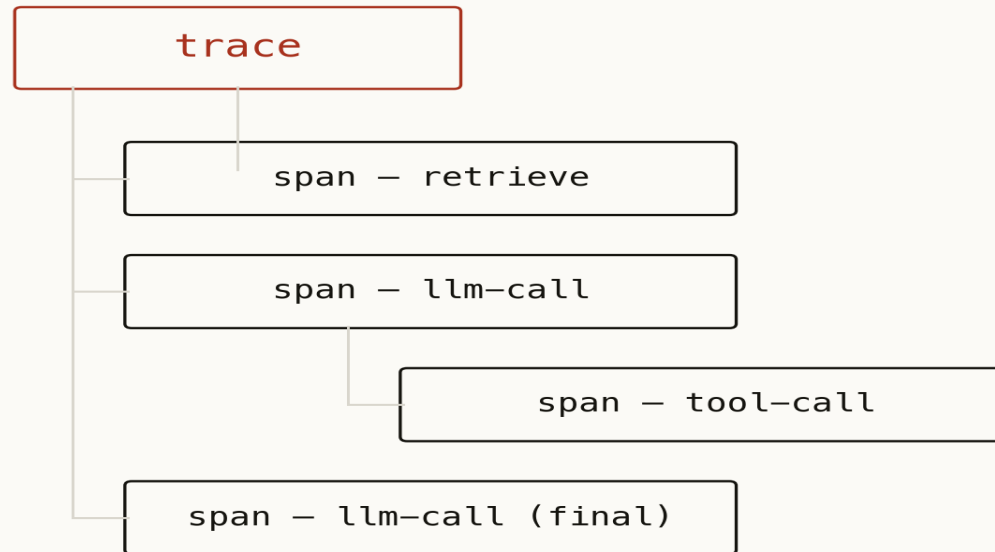
One operation inside a trace.

LLM call • retrieval • tool • custom logic.

The tree.

The trace tree

Parent to child = causality.



Parent → child = causality.

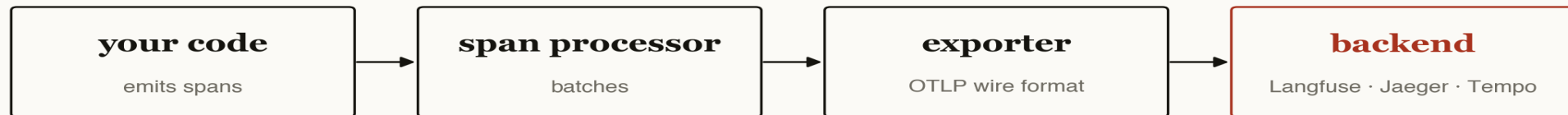
Attributes.

- Key / value metadata on a span.
-
- user · session · cost · latency · tags.
-
- Filtering later depends on this.

The pipeline.

The pipeline

OpenTelemetry is the wire format — everything downstream is swappable.



Jaeger · Tempo · Datadog · Langfuse — all speak OTLP.

OpenTelemetry is the wire format.

Why this matters.

— Jaeger · Tempo · Datadog · Langfuse.

— All speak OTLP.

— Learn the concept, swap the tool.

The LLM-specific piece.

Generation

a span with first-class fields

model

which model ran

prompt / completion

the text in and out

token usage

the cost driver

Part 3 — Raw OTel's Limits

What raw OTel can't do.

Structured and causal — but not readable for LLM work.

We just ran raw OTel.

- Structured? yes.
- Causal? yes.
- Readable for LLM work? no.
- (JSON blobs, hex parent IDs.)

The gaps.

- No cost — there are no price tables.
- No prompt / completion rendering.
- No place for a score.
- No sessions, users, or UI.

So why doesn't APM just work?

- Datadog / Jaeger — generic spans.
- No token concept.
- No prompt $\leftrightarrow$ completion pairing.
- $\rightarrow$ an LLM-native layer must exist.

Part 4 — Why Providers Exist

Why providers exist.

Not alternatives to OTel — backends for it.

Not alternatives to OTel.

*Providers are OTel backends —
with LLM-aware storage & UI on top.*

The landscape.

Langfuse

OSS · OTel-native · self-hostable

LangSmith

tightest LangChain fit

Phoenix (Arize)

eval / experiment focus

Helicone

proxy · near-zero setup

Why Langfuse here.

- Open source.
- OTel-native – LangfuseSpanProcessor.
- Broad framework support.
- The concepts transfer to all of them.

Part 5 — Langfuse, Live

Langfuse.

The reveal.

LangfuseSpanProcessor

= the span processor from Part 2.

Part 2 was the architecture, not an analogy.

Three ways to instrument.

- 01 Drop-in: `from langfuse.openai import openai`
- 02 Decorator: `@observe()`
- 03 Context manager: `start_as_current_observation`

The trace UI.

- Tree view.
-
- Per-span latency + cost.
-
- Filter by user / session / tag.

Part 6 — Closing the Loop

Closing the loop.

Scores on live traffic, not a static set.

Evals, again.

- The evals class: a static set.
-
- Now: scores on live traffic.

Score the trace.

LLM-as-judge

→ *attach to trace*

Heuristic

→ *attach to trace*

Human review

→ *attach to trace*

Why observability matters.

*Regression becomes visible —
before users hit it.*

Takeaways.

- Trace · span · generation.
-
- OTel is the substrate.
-
- Observability + evals = a feedback loop.